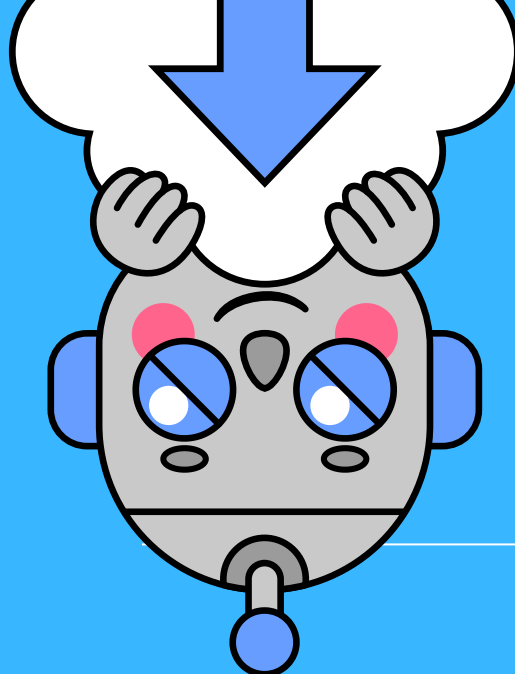


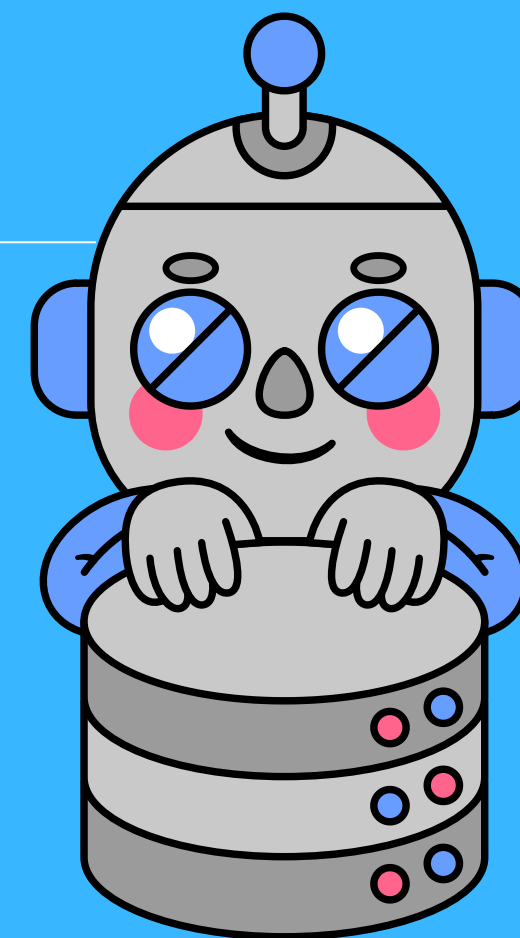
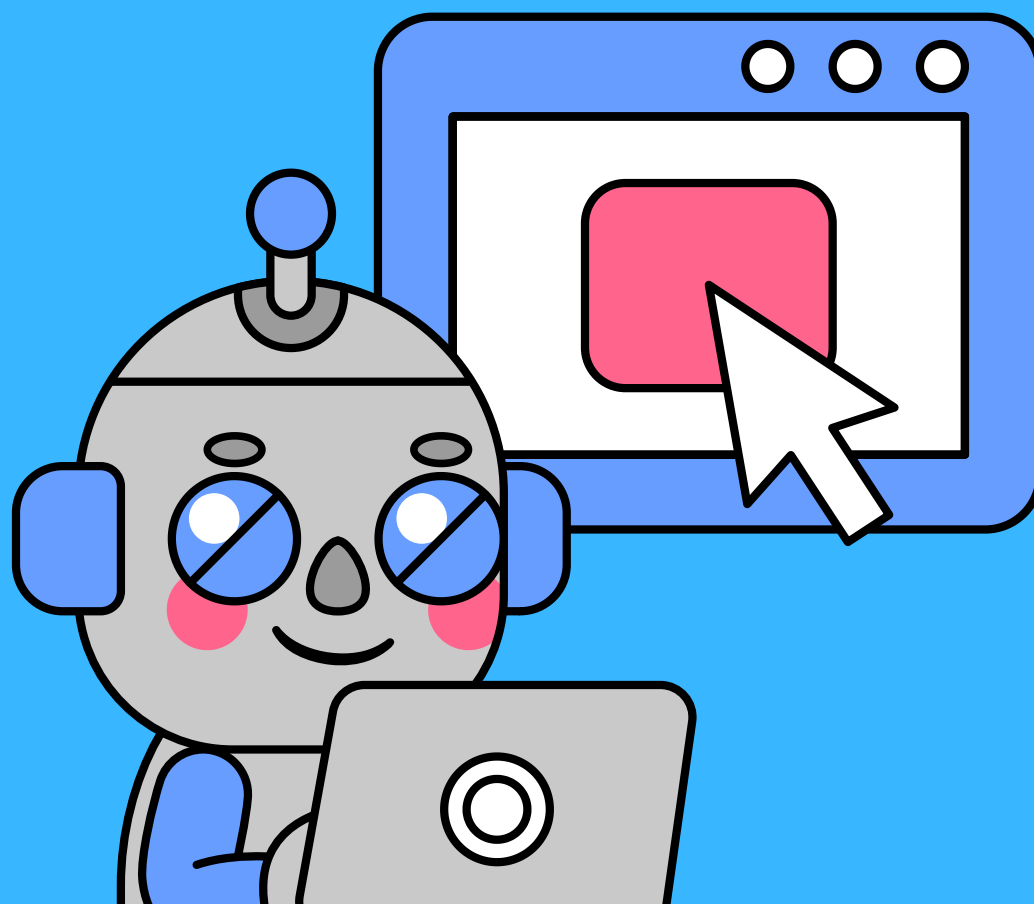


Artificial Intelligence

인공지능 기초

인공지능은 어떻게 스스로
똑똑해질까?

Chapter 03. 기계학습과 데이터(4)



1 인공지능 모델

1. 선형 회귀 - 회귀
2. 의사결정트리 - 회귀/ 분류
3. K최근접 이웃 - 회귀/ 분류

1 인공지능 모델

모든 인공지능 모델의 생성 과정

1. 문제(X)와 정답(Y) 준비
2. 훈련 데이터와 테스트 데이터로 분할하기
3. 인공지능 모델 생성하기(빈 모델)
4. 모델에 훈련 데이터로 학습 시키기
5. 테스트 데이터로 예측하기
6. 정확도 평가하기
7. 실제 예측 데이터(X)를 넣고 예측하기

2 선형회귀

1. 선형 회귀의 이해

공부 시간을 입력하면 성적을 예측할 수 있을까?

선형 (Linear)

데이터들 사이의 관계가 '직선'의 형태를 띠는 것을 의미합니다. 즉, 하나의 값이 커질 때 다른 값도 일정한 비율로 커지거나 작아지는 성질입니다.

회귀 (Regression)

영국의 유전학자 프랜시스 골턴이 부모와 자식의 키를 연구하며 "세대가 지날수록 결국 인간의 '평균적인 키'로 돌아가려는(회귀하려는) 경향이 있다"고 발표한 데서 유래한 용어입니다.

수많은 데이터들이 흩어져 있을 때, 이 데이터들을 가장 잘 설명할 수 있는

단 하나의 최적화된 직선을 긋고

그 선을 따라 평균적인 경향성을 파악해 미지의 값을 예측하는 기법

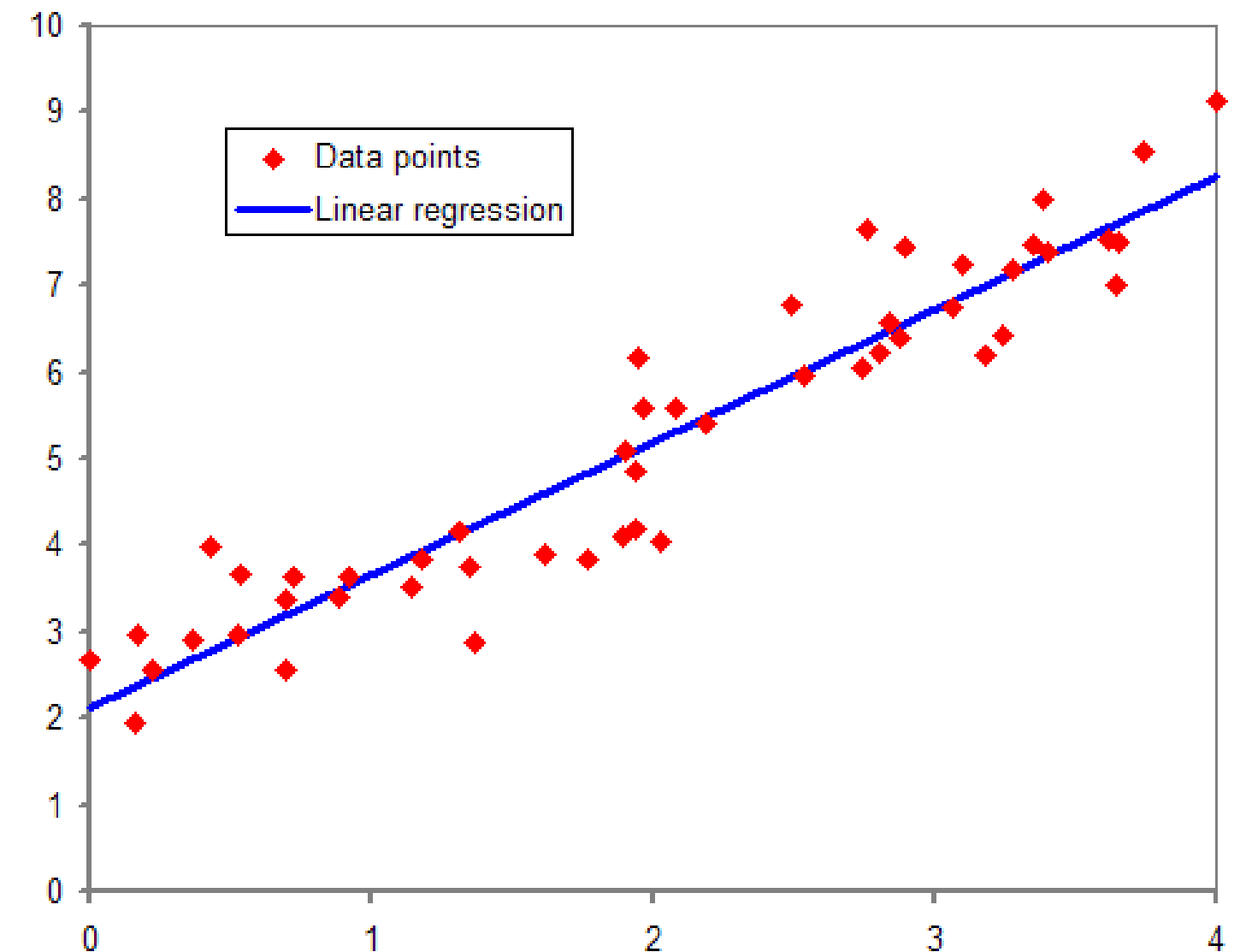
2 선형회귀

1. 선형 회귀의 이해

수많은 데이터들이 흩어져 있을 때, 이 데이터들을 가장 잘 설명할 수 있는

단 하나의 최적화된 직선을 긋고

그 선을 따라 평균적인 경향성을 파악해 미지의 값을 예측하는 기법



2 선형회귀

1. 선형 회귀의 이해 - 독립 변수와 종속 변수

선형회귀는 지도학습 방법이므로, 인공지능에게 문제와 정답을 함께 알려주어야 함.

문제 → '원인' / 정답 → '결과'

독립 변수 (x)

"내 마음대로 변할 수 있는 원인"

다른 것에 영향을 받지 않고 스스로 독립적으로 존재하거나 조작할 수 있는 값입니다. 예측을 위해 우리가 인공지능에게 **입력(Input)**해 주는 데이터입니다.

예시 하루 스마트폰 사용 시간, 아이스 아메리카노 가격, 코딩 연습 시간

종속 변수 (y)

"원인에 따라 결정되는 결과"

독립 변수(x)가 어떻게 변하느냐에 따라 값이 종속적으로 결정됩니다. 우리가 최종적으로 알고 싶어 하는 **정답(Label)**이자 **출력(Output)** 데이터입니다.

예시 남은 배터리 용량, 커피 판매량, 해커톤 대회 점수

2 선형회귀

1. 선형 회귀의 이해 - 독립 변수와 종속 변수

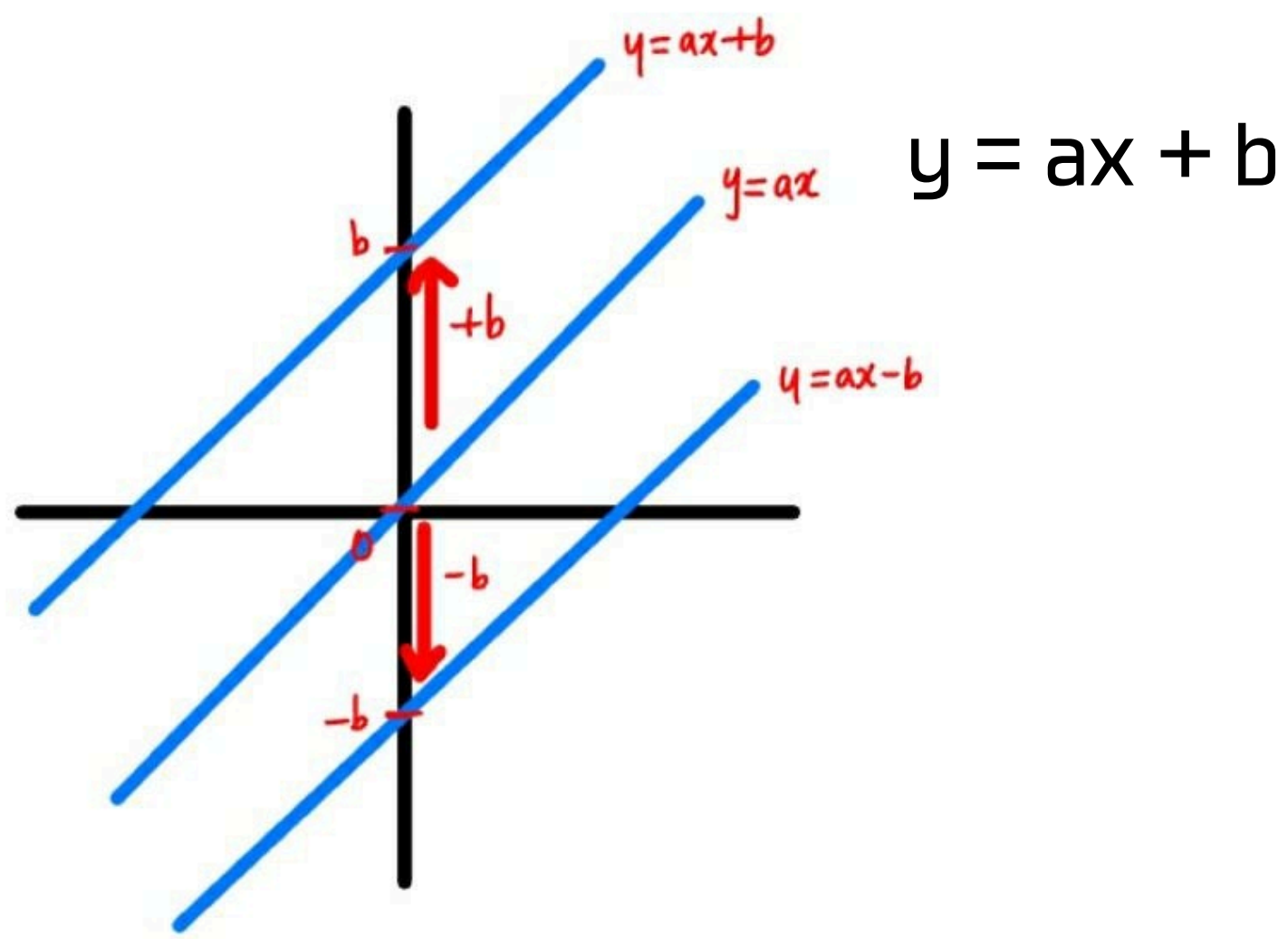
선형회귀는 지도학습 방법이므로, 인공지능에게 문제와 정답을 함께 알려주어야 함.

문제 → '원인' / 정답 → '결과'

- 공부 시간에 따른 시험 성적 예측
- 기온에 따른 아이스크림 판매량 예측
- 아파트 평수와 매매 가격
- 자동차 주행 거리와 중고차 가격
- 광고비 지출에 따른 웹 사이트 방문자 수 예측

2 선형회귀

1. 선형 회귀의 이해 - 선형 회귀식



가중치 편향

$y = wx + b$

결과 원인

↔ x 와 y (입력과 예측)

x는 우리가 이미 알고 있는 원인 데이터(예: 공부 시간)이고, **y**는 우리가 알고 싶은 최종 결과(예: 예상 시험 점수)입니다.

📦 w (가중치, Weight)

"x가 y에 얼마나 강력한 영향을 미치는가?" (기울기)

- **w=10** : 공부를 1시간 할 때마다 점수가 10점씩 오름.
- **w=1** : 공부를 1시간 해도 점수가 1점밖에 오르지 않음.

즉, w는 원인이 결과에 미치는 '영향력의 크기'입니다.

🏠 b (편향, Bias)

"아무것도 안 했을 때의 기본값은 얼마인가?" (y절편)

공부 시간(x)이 0시간일 때 무조건 0점을 받는 것이 아니라, 번호를 무작위로 찍어서 기본적으로 20점을 받을 수 있다면 이때의 b는 20이 됩니다. 즉, 데이터가 기본적으로 어느 위치에서 시작하는지를 잡아주는 **영점 조절기** 역할을 합니다.

2 선형회귀

1. 선형 회귀의 이해 - 선형 회귀의 학습 과정

인공지능 모델을 '학습(Training)'시킨다는 것은, 결국 데이터들을 가장 잘 대표하는 완벽한 w (가중치)와 b (편향)를 컴퓨터가 스스로 찾아내는 과정입니다.

1

무작위 선 긋기

처음에 인공지능은 아무것도 모르기 때문에 허공에 엉뚱한 직선(무작위 w, b 값)을 대충 그어봅니다.

2

오차(Error) 계산

대충 그은 선의 예측값과 실제 데이터(진짜 점수) 사이의 거리 차이(오차)가 얼마나 큰지 계산합니다.

3

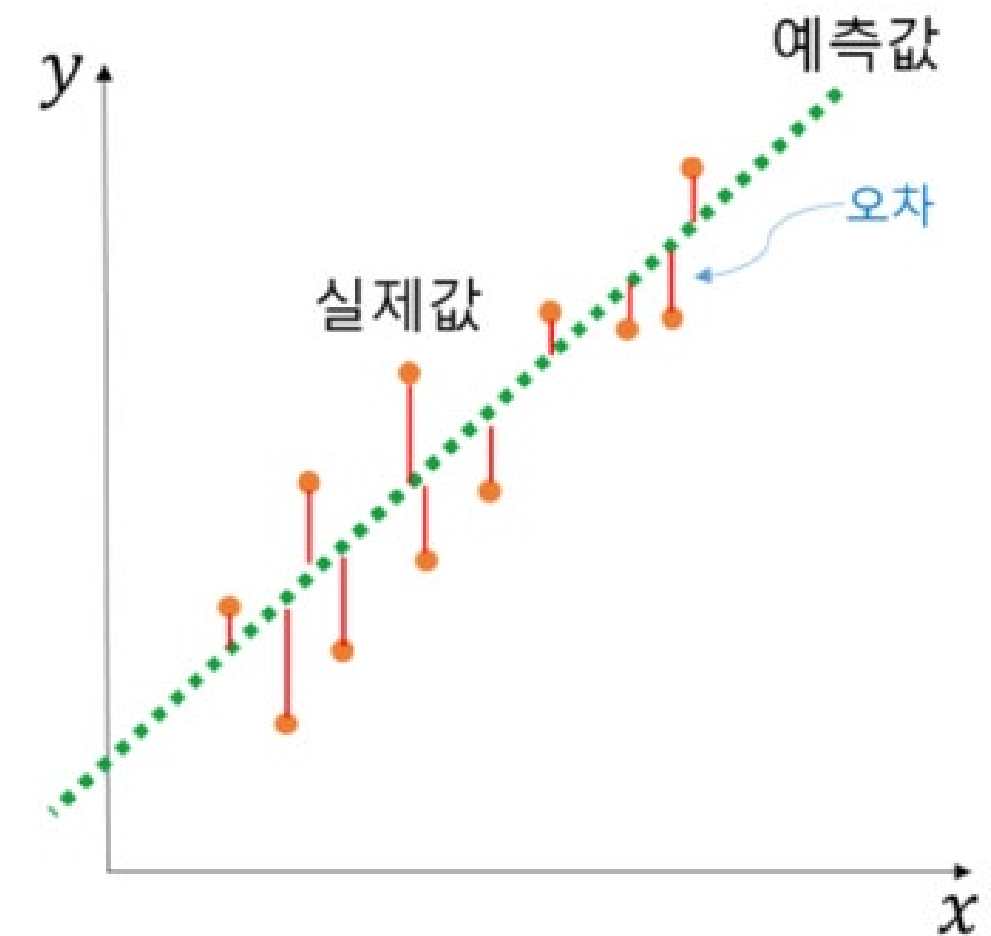
선 수정(업데이트)

오차가 너무 크면 선의 기울기(w)를 높히거나 세워보고, 시작점(b)을 위아래로 움직여 선을 수정합니다.

4

최적의 선 완성

이 과정을 엄청나게 빠르게 수천 번 반복하여 '오차가 가장 작아지는 지점'을 확정합니다. 이것이 학습 완료입니다!



2 선형회귀

1. 선형 회귀의 구현 - 사이킷런

scikit-learn (사이킷런)

파이썬에서 머신러닝을 쉽게 할 수 있도록 지원하는 라이브러리
(분류, 회귀, 군집, 모델 생성 및 평가)

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.model_selection import train_test_split
```

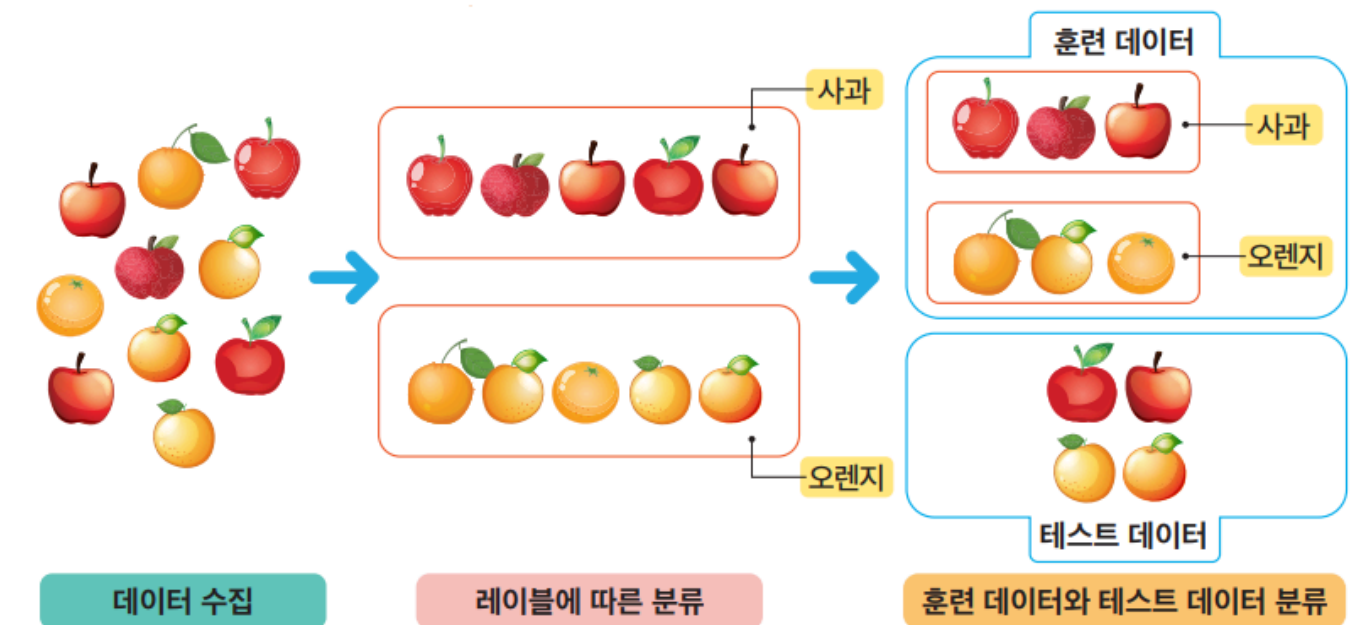
2 선형회귀

1. 선형 회귀의 구현 - train_test_split 핵심 함수

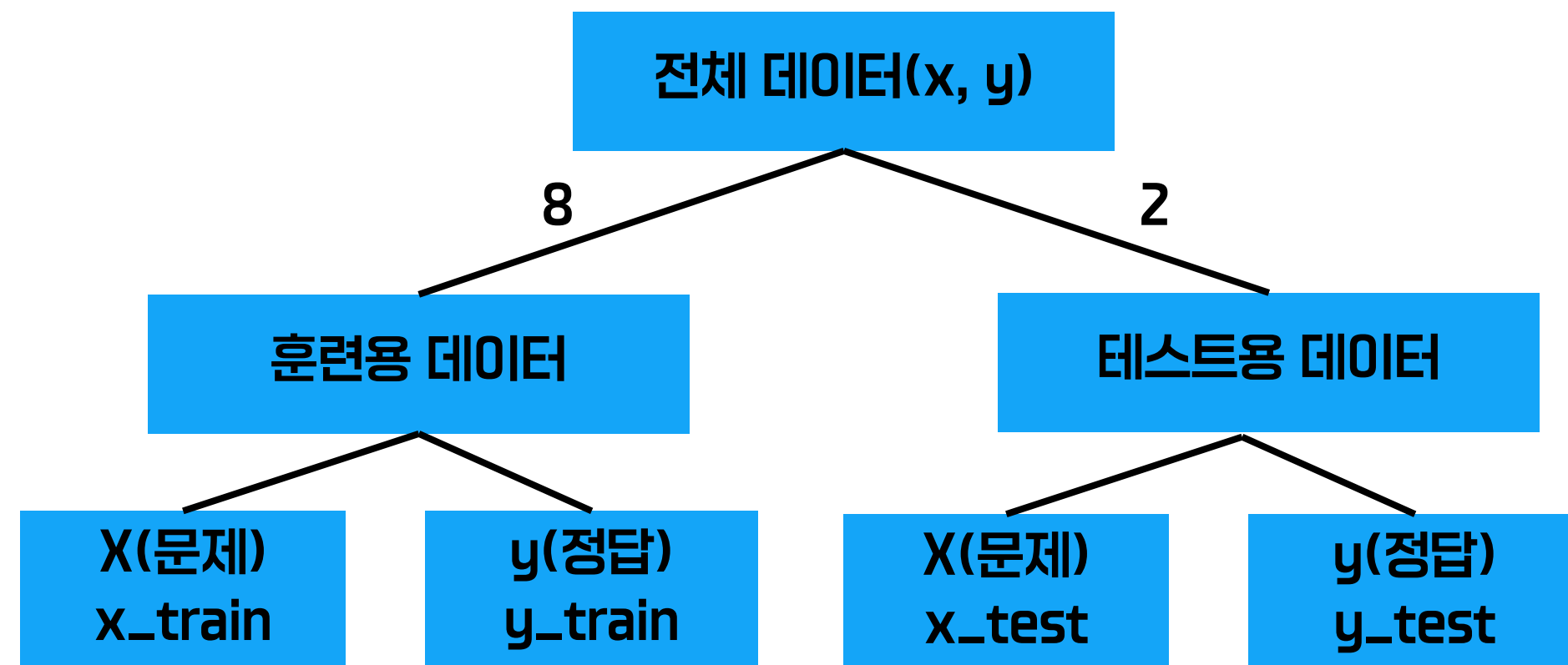
✂ 1. 데이터 분할 도구 (sklearn.model_selection)

모델이 특정 훈련 데이터에만 맞춰지는 '과적합(Overfitting)' 현상을 방지하기 위해, 전체 데이터를 학습용(훈련)과 평가용(테스트)으로 무작위로 나누는 필수적인 과정입니다.

- `train_test_split(X, y, test_size, random_state)` : 데이터를 무작위 비율로 분할합니다.



`x_train, x_test, y_train, y_test = train_test_split(문제, 정답, test_size=0.2, random_state=42)`



2 선형회귀

1. 선형 회귀의 구현 - sklearn.linear_model 핵심 함수

⚙️ 2. 선형 모델 도구 (sklearn.linear_model)

선형 회귀 알고리즘을 실제로 생성하고, 학습시키고, 예측하는 핵심 도구들입니다.

LinearRegression() 비어있는 선형 회귀 모델 객체를 생성합니다.

fit(X_train, y_train) 훈련용 문제와 정답을 모델에 입력하여 최적의 선을 찾는 **학습**을 수행합니다.

predict(X_test) 학습이 완료된 모델에 새로운 문제 데이터를 입력하여 결과를 **예측**합니다.

score(X_test, y_test) 테스트 데이터를 바탕으로 모델 성능을 **평가**합니다. 예측의 정확도를 나타내는 '결정계수(R^2)'를 반환하며, 1에 가까울수록 우수합니다.

2 선형회귀

1. 선형 회귀의 구현 - sklearn.linear_model 핵심 함수

```
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression
```

1. 데이터 분할: 전체 데이터를 훈련용(80%)과 테스트용(20%)으로 무작위로 나눕니다.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2. 모델 생성: 비어있는 선형 회귀 모델을 준비합니다.

```
model = LinearRegression()
```

3. 모델 학습: 훈련 데이터로 최적의 선(w, b)을 찾습니다.

```
model.fit(X_train, y_train)
```

4. 결과 예측: 학습된 모델에 테스트 문제를 주어 예측값을 받습니다.

```
predictions = model.predict(X_test)
```

5. 성능 평가: 모델이 얼마나 똑똑한지 평가 점수(1에 가까울수록 완벽)를 확인합니다.

```
print(model.score(X_test, y_test))
```

3 의사결정트리

1. 의사결정트리의 이해

스무고개로 정답을 찾는 인공지능

③ 질문으로 데이터 쪼개기

데이터가 가진 여러 가지 특징(속성)을 보고, "날개가 있나요?", "공부 시간이 4시간 미만인가요?"와 같이 '예/아니요'로 대답할 수 있는 조건을 만듭니다.

📍 경로를 따라 이동하기

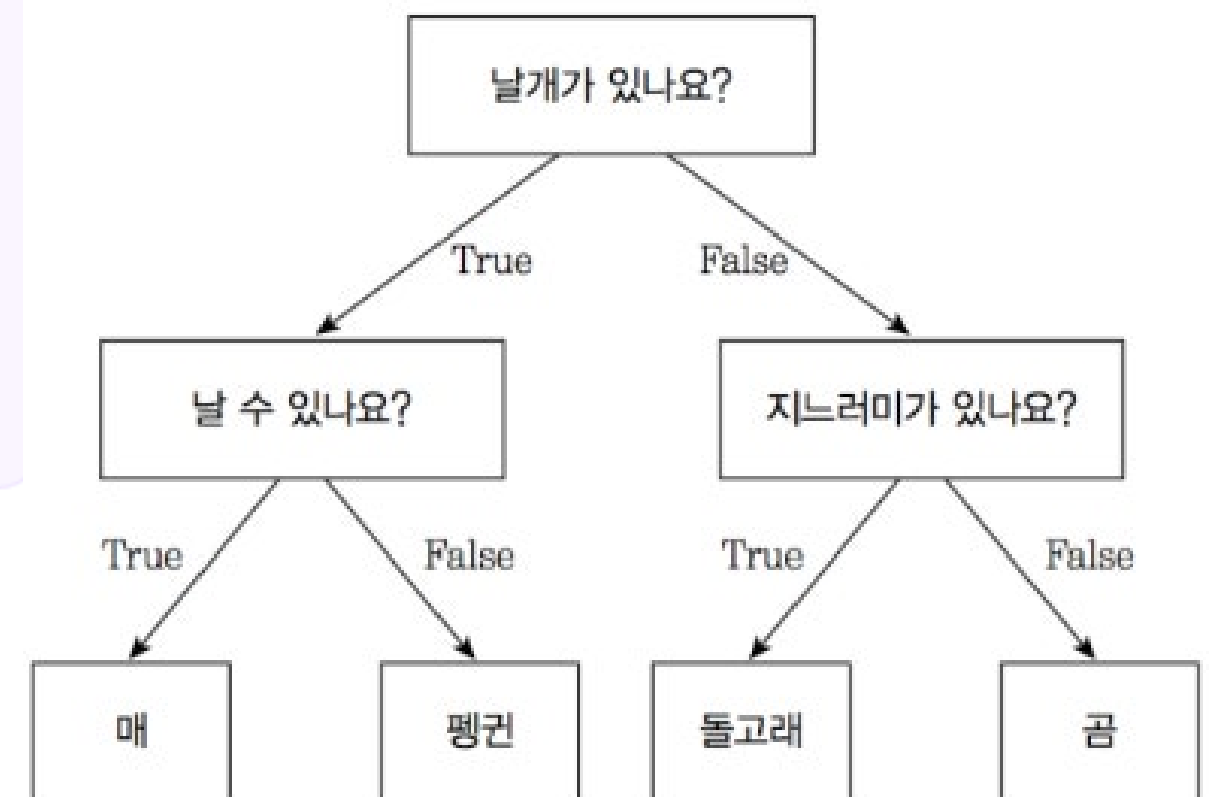
질문에 대한 대답(참/거짓)에 따라 정해진 경로를 따라 이동하며, 최종적으로 도달한 끝에서 정답을 내립니다.

데이터 속성을 기준(질문)을 만들고,

True or False 에 따라

나무가 가지를 뻗어나가듯 이동 경로를 결정하여 예측하는 방법

(주로 분류 문제에 사용하지만 회귀(예측)에도 활용 가능)



3 의사결정트리

1. 의사결정트리의 이해

의사결정트리는 지도학습 방법이므로, 인공지능에게 문제와 정답을 함께 알려주어야 함.

문제 → '원인'(독립변수) / 정답 → '결과'(종속변수)

- 소득 수준, 직업 등 금융 정보를 바탕으로 대출 승인 여부 분류
- 광고 단어, 첨부파일 여부, 링크 포함 여부 등을 바탕으로 스팸 메일 분류
- 꽃잎 길이, 너비, 색상등을 바탕으로 식물 종류 분류
- 비만 여부, 운동 여부, 가족력 등 환자 데이터를 바탕으로 당뇨병 위험군 분류

3 의사결정트리

1. 의사결정트리의 이해

[독립 변수] : 비, 강한 바람, 더움

[종속 변수] : 활동

의사결정트리 인공지능 모델은 데이터를 분석하여 어떤 속성을 어떻게 선택하는 것이 가장 효율적인지 지능적 판단을 함.

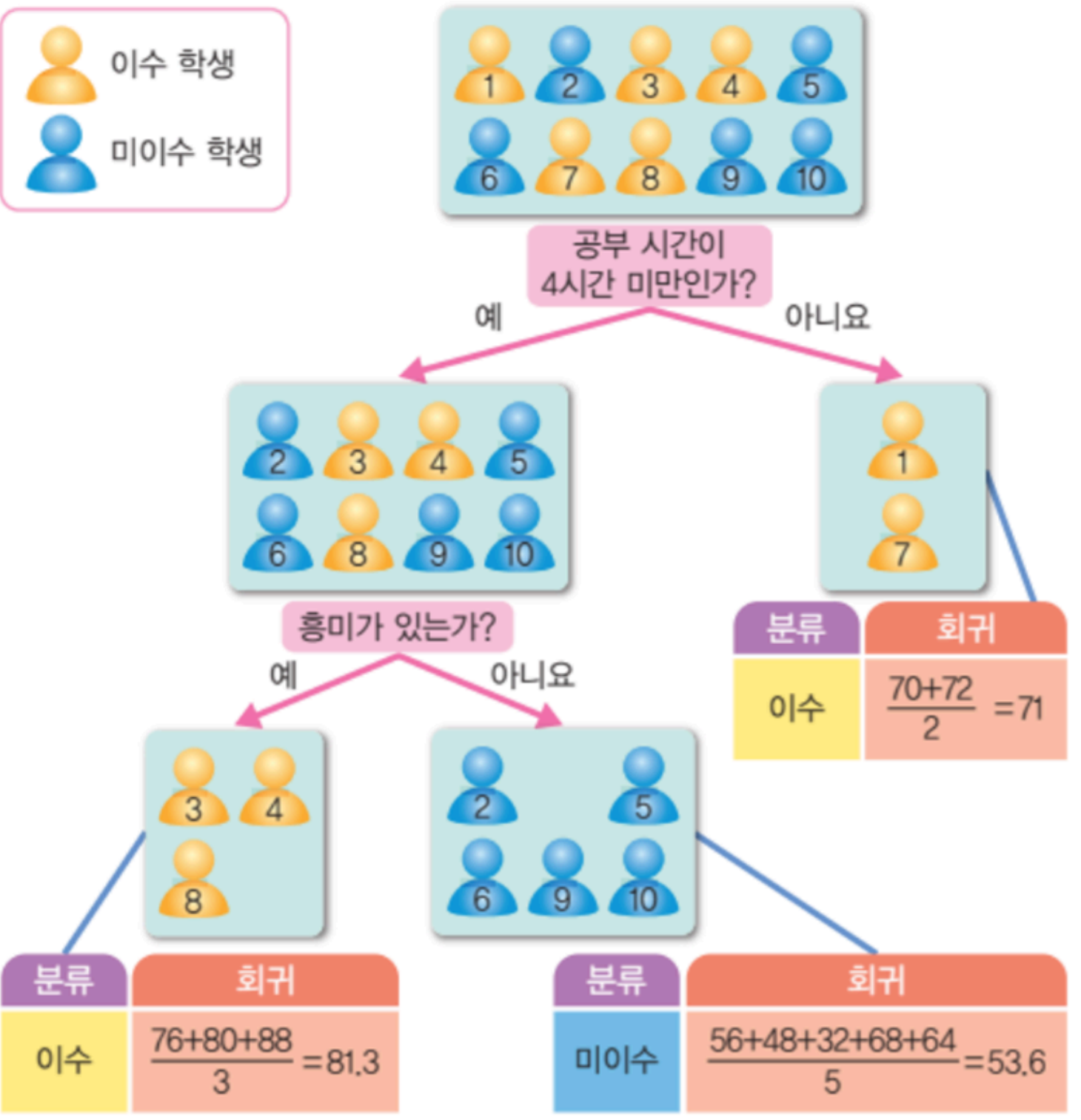
분할 결과가 가능하면 동질적으로 만들어지도록

비	강한 바람	더움	활동
0	0	1	야외 활동
0	0	0	야외 활동
0	1	1	실내 활동
1	0	1	실내 활동
1	1	0	실내 활동

3 의사결정트리

1. 의사결정트리의 이해

번호	공부 시간	흥미	점수	이수/미이수
1	5	아니요	70	이수
2	1	아니요	56	미이수
3	2	예	76	이수
4	3	예	80	이수
5	2	아니요	48	미이수
6	3	아니요	32	미이수
7	4	예	72	이수
8	2	예	88	이수
9	1	아니요	68	미이수
10	2	아니요	64	미이수



3 의사결정트리

1. 의사결정트리의 이해

"질문을 계속해서 쫓아다 보면 결국 완벽한 모델이 되지 않을까요?"
정답은 '아니요' 입니다! 여기에 아주 헛갈리기 쉬운 함정이 숨어있습니다.

💡 나무가 너무 깊어지면 생기는 문제

나무의 깊이(질문의 수)가 너무 깊어지면, 모델이 연습 문제로 주어지는 '**훈련 데이터**'에만 지나치게 **집착하여** 아주 지엽적인 규칙까지 모두 외워버리게 됩니다.

이 상태를 **과적합(Overfitting)**이라고 부릅니다. 과적합이 발생하면, 막상 실전 시험인 '**새로운 데이터**'가 주어졌을 때는 유연하게 대처하지 못해 예측 정확도가 크게 떨어지게 됩니다. 따라서 의사결정트리를 쓸 때는 질문의 최대 개수(나무의 깊이)를 적절히 제한해 주는 것이 필수적입니다!

3 의사결정트리

2. 의사결정트리의 구현 - 사이킷런

scikit-learn (사이킷런)

파이썬에서 머신러닝을 쉽게 할 수 있도록 지원하는 라이브러리

(분류, 회귀, 군집, 모델 생성 및 평가)

```
from sklearn.tree import DecisionTreeClassifier
```

3 의사결정트리

2. 의사결정트리의 구현

```
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier
```

1. 훈련/테스트 데이터로 분할

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2. 모델 생성

```
model = DecisionTreeClassifier()
```

3. 모델 학습 (스스로 질문 규칙 나무 만들기)

```
model.fit(X_train, y_train)
```

4. 새로운 데이터 예측하기

```
predictions = model.predict(X_test)
```

5. 성능 평가하기

```
print(model.score(X_test, y_test))
```

3 의사결정트리

1. 의사결정트리의 구현

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# 1. 문제(x)와 정답(y) 분리하기
x = df[["rain", "strong_wind", "hot"]] # 날씨 조건 (입력값)
y = df["activity"] # 할 활동 (우리가 맞힐 정답)

# 2. 데이터를 훈련용과 테스트용으로 나누기
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

# 3. 의사결정트리 모델 생성 (나무가 너무 깊어지는 것을 막기 위해 max_depth 옵션을 줍니다. )
model = DecisionTreeClassifier(max_depth=3)

# 4. 모델 학습시키기 (최적의 질문 찾기)
model.fit(x_train, y_train)

# 5. 새로운 날씨에 대한 활동 예측하기
predictions = model.predict(x_test)
print(predictions)
```

4 K최근접 이웃 알고리즘(K-Nearest Neighbors, KNN 알고리즘)

1. K최근접 이웃(KNN)의 이해

가까운 이웃을 보고 정답을 유추하는 분류 방법

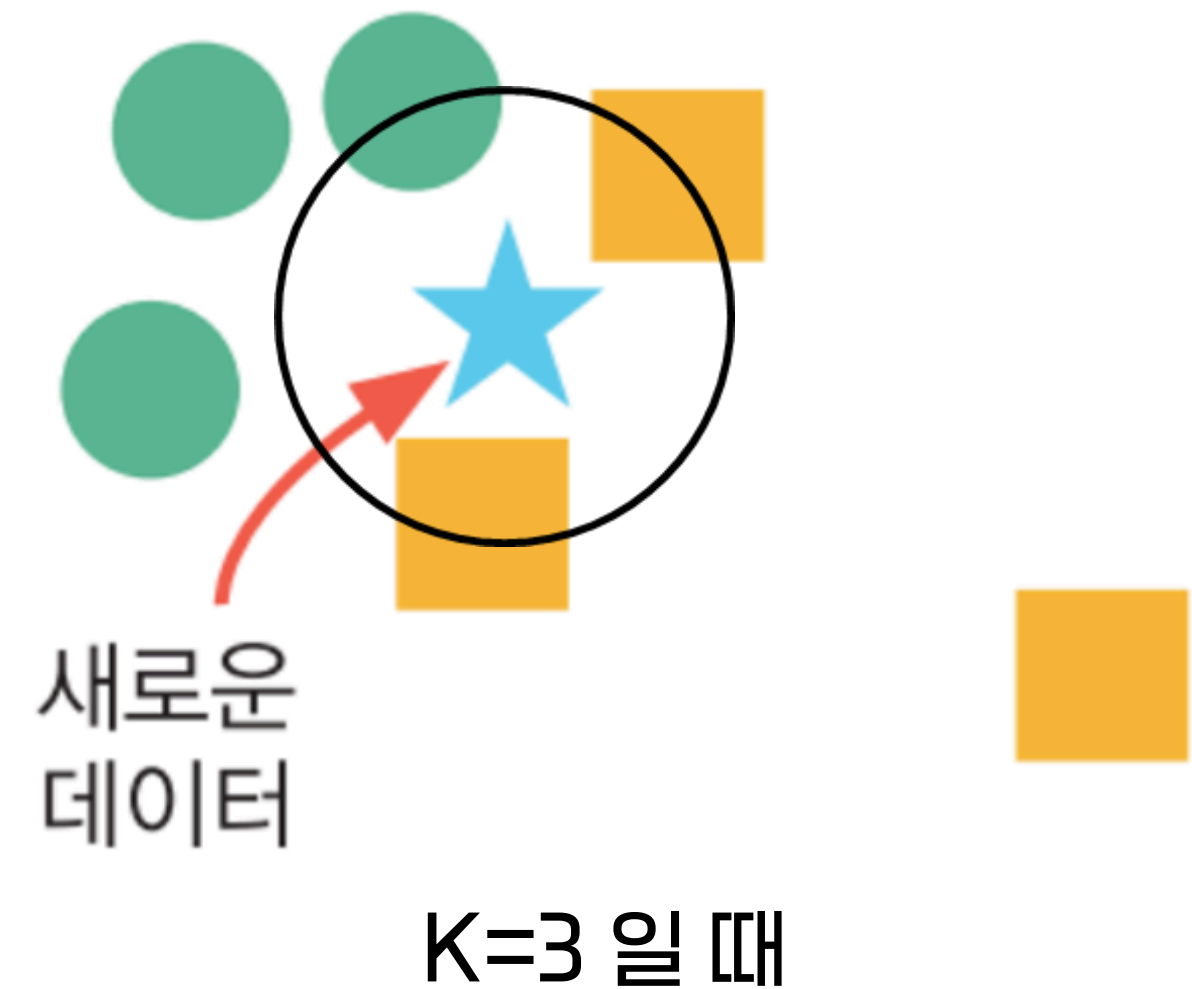
👥 이웃에게 물어보기

새로운 데이터가 주어졌을 때, 컴퓨터는 이 데이터 주변에 **가장 가까이 있는 기존 데이터들 (이웃)**이 어떤 종류인지 살펴봅니다.

🗳️ 다수결의 원칙

가장 가까운 이웃들이 속한 그룹을 확인하고, **다수결 투표**를 통해 새로운 데이터가 어디에 속할지 최종 결정합니다.

새로운 질문이 들어오면,
기존 데이터 중 **가장 가까운 K개의 이웃**을 찾아
그 이웃들이 많이 가지고 있는 정답을 예측하는 기법



4 K최근접 이웃 알고리즘(K-Nearest Neighbors, KNN 알고리즘)

1. K최근접 이웃(KNN)의 이해

KNN 모델에서 개발자가 유일하게 신경 써야 할 것은 '몇 명의 이웃(K)에게 물어볼 것인가?'를 정해주는 것입니다. K값에 따라 인공지능의 성능이 완전히 달라집니다.

1 K = 1 일 때

가장 가까운 단 1명의 이웃만 보고 판단합니다.
아주 사소한 데이터나 오타(노이즈)에도 극도로 민감하게 반응하여 판단이 오락가락할 수 있습니다.

K 가 너무 클 때 (예: K=100)

주변의 특징을 세밀하게 보지 않고 무조건 전체 그룹에서 숫자가 많은 쪽으로만 예측해버립니다. 세밀한 예측이 불가능해집니다.

💡 주의!: K는 보통 '홀수'로 설정합니다!

만약 합격/불합격을 나누는 문제에서 K를 4로 설정했는데, 이웃이 합격 2명, 불합격 2명으로 동점이 나오면 판단을 내릴 수 없습니다. 따라서 동점을 방지하기 위해 보통 3, 5, 7과 같은 홀수로 설정합니다.

4 K최근접 이웃 알고리즘(K-Nearest Neighbors, KNN 알고리즘)

1. K최근접 이웃 알고리즘의 이해

K최근접 이웃(KNN) 지도학습 방법이므로, 인공지능에게 문제와 정답을 함께 알려주어야 함.

문제 → '원인'(독립변수) / 정답 → '결과'(종속변수)

- 영화 취향에 맞춘 영화 장르 추천
- 나이, 구매 금액, 구매 횟수 등 구매 데이터를 바탕으로 상품 추천
- 공부시간, 독서량 등 학습 패턴을 바탕으로 유사한 학습 유형 학생 그룹 분류

4 K최근접 이웃 알고리즘(K-Nearest Neighbors, KNN 알고리즘)

1. K최근접 이웃(KNN)의 구현 - 사이킷런

scikit-learn (사이킷런)

파이썬에서 머신러닝을 쉽게 할 수 있도록 지원하는 라이브러리
(분류, 회귀, 군집, 모델 생성 및 평가)

```
from sklearn.neighbors import KNeighborsClassifier
```

```
KNeighborsClassifier(n_neighbors=K)
```

K개의 이웃을 살펴보는 '분류' 모델을 만듭니다. (예: 암 양성/음성 예측)

4 K최근접 이웃 알고리즘(K-Nearest Neighbors, KNN 알고리즘)

1. K최근접 이웃(KNN)의 구현 - 사이킷런

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
# 1. 문제(X: 무게, 당도)와 정답(y: 과일 이름) 분리
```

```
X = df[['무게', '당도']]
```

```
y = df['과일종류']
```

```
# 2. 훈련용/테스트용 데이터 분할
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# 3. 모델 생성: 주변 3개의 이웃을 확인하는 모델
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
# 4. 모델 학습
```

```
model.fit(X_train, y_train)
```

```
# 5. 새로운 과일 예측하기
```

```
predictions = model.predict(X_test)
```

```
print(model.score(X_test, y_test))
```